

ETC5513 Assignment 2: The Guide

Shravani Jadhav

Introduction

This guide demonstrates my knowledge of git, GitHub, and the command line interface for version control and collaboration. Each step explains the best practices for reproducible and collaborative workflows. All commands were run using Git Bash on Windows.

Step 1: Create a New RStudio Project and Quarto File

I created a new RStudio project called ETC5513_Assignment2 and created a Quarto document file called example.qmd. In the document, I added a YAML header (including the title, author, and HTML format), introduction section, and R code chunk that runs `summary(cars)` to produce a basic summary table.

A Quarto document combines text and executable R code in one file. I rendered the document to a HTML file which is shown below.

ETC5513 Assignment 2

AUTHOR

Shravani Jadhav

Introduction

This is a simple Quarto document created for ETC5513 Assignment 2.

Summary Statistics

Below is a summary of the `cars` dataset.

```
summary(cars)
```

speed	dist
Min. : 4.0	Min. : 2.00
1st Qu.:12.0	1st Qu.: 26.00
Median :15.0	Median : 36.00
Mean :15.4	Mean : 42.98
3rd Qu.:19.0	3rd Qu.: 56.00
Max. :25.0	Max. :120.00

The dataset contains 50 observations.

Figure 1: Rendered HTML output of example.qmd

Step 2: Initialise as a Git Repository and Push to GitHub

In Git Bash using the command `cd`, I changed my current directory to the assignment folder and initialised it as a local git repository with the use of `git init`. Then, I staged all the files by `git add .` and committed them all using `git commit -m "Initial commit: add example.qmd and RStudio project"`

Then I created a repository in GitHub named `ETC5513_Assignment2`. By using the commands `git remote add origin` (along with the SSH key from GitHub) and `git push`, I was able to connect

my local repository to GitHub. The codes and the screenshots that demonstrate the process are shown below.

```
shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git remote add origin git@github.com:sjad0015-git/ETC5513_Assignment2.git

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git push -u origin main
Enumerating objects: 56, done.
Counting objects: 100% (56/56), done.
Delta compression using up to 12 threads
Compressing objects: 100% (44/44), done.
Writing objects: 100% (56/56), 363.83 KiB | 2.91 MiB/s, done.
Total 56 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To github.com:sjad0015-git/ETC5513_Assignment2.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git add .

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git commit -m "added step1 image to images folder"
[main 31d088b] added step1 image to images folder
1 file changed, 0 insertions(+), 0 deletions(-)
rename Step1.jpg => images/Step1.jpg (100%)

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 335 bytes | 111.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:sjad0015-git/ETC5513_Assignment2.git
 6909860..31d088b  main -> main
```

Figure 2: Terminal showing successful push to GitHub

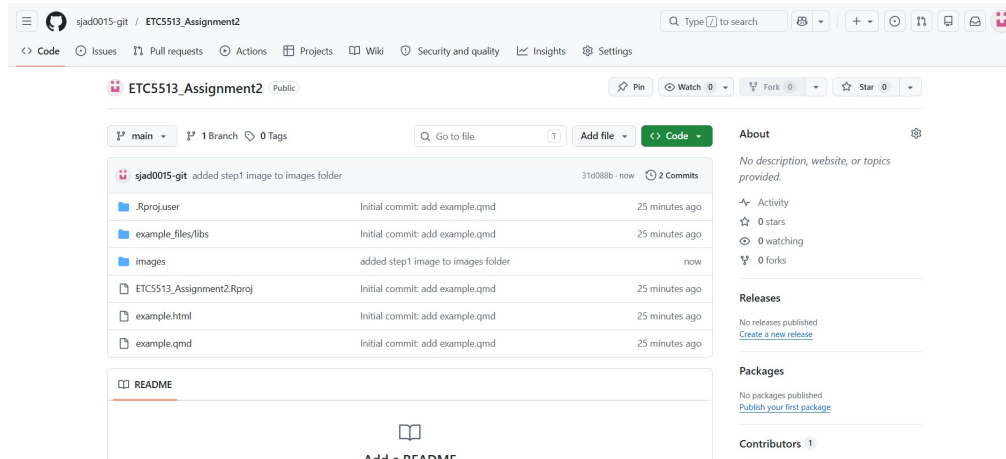


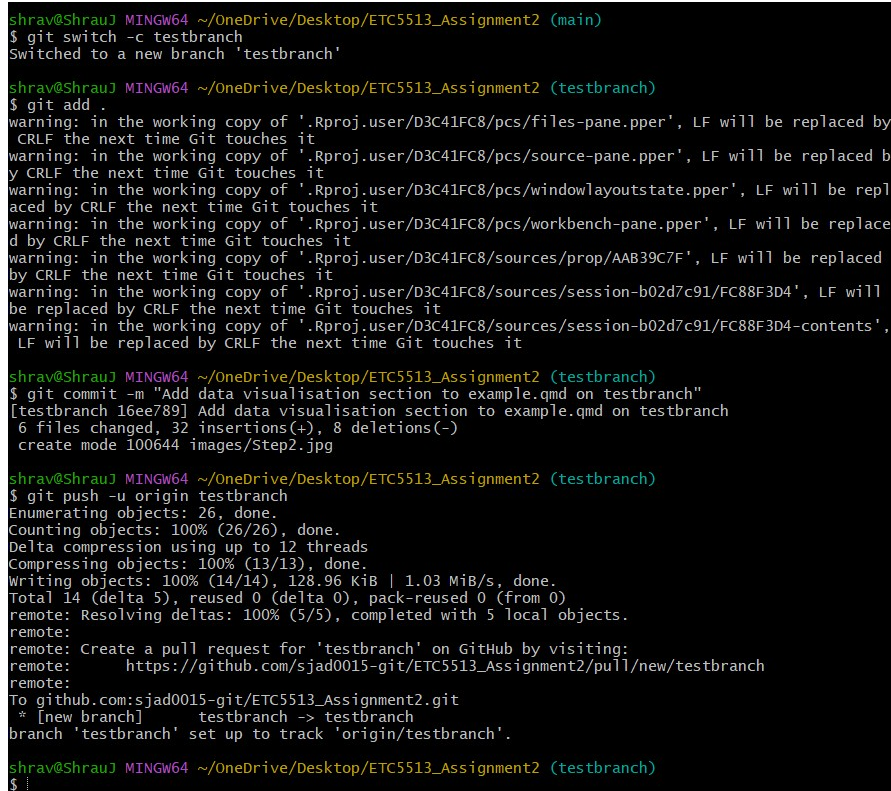
Figure 3: GitHub repository showing project files

Step 3: Create testbranch and Modify example.qmd

I created a new branch named testbranch and also switched to it at once, using `git switch -c testbranch`. Now, I can make changes separately on this branch without affecting the main branch until required. On the new branch, I created a section on data visualization with a scatter plot of the cars dataset in `example.qmd` file. Then, I added, committed and pushed my changes to the remote repository as we did in Step 2, just this time in the testbranch.

```
git add .
git commit -m "Add data visualisation section to example.qmd on testbranch"
git push -u origin testbranch
```

The screenshots below depict the success of push to GitHub.



```
shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git switch -c testbranch
Switched to a new branch 'testbranch'

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (testbranch)
$ git add .
warning: in the working copy of '.Rproj.user/D3C41FC8/pcs/files-pane.pper', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/pcs/source-pane.pper', LF will be replaced b
y CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/pcs/windowlayoutstate.pper', LF will be repl
aced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/pcs/workbench-pane.pper', LF will be replace
d by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/prop/AAB39C7F', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/session-b02d7c91/FC88F3D4', LF will
be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/session-b02d7c91/FC88F3D4-contents',
LF will be replaced by CRLF the next time Git touches it

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (testbranch)
$ git commit -m "Add data visualisation section to example.qmd on testbranch"
[testbranch 16ee789] Add data visualisation section to example.qmd on testbranch
6 files changed, 32 insertions(+), 8 deletions(-)
create mode 100644 images/Step2.jpg

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (testbranch)
$ git push -u origin testbranch
Enumerating objects: 26, done.
Counting objects: 100% (26/26), done.
Delta compression using up to 12 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (14/14), 128.96 KiB | 1.03 MiB/s, done.
Total 14 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
remote:
remote: Create a pull request for 'testbranch' on GitHub by visiting:
remote:   https://github.com/sjad0015-git/ETC5513_Assignment2/pull/new/testbranch
remote:
To github.com:sjad0015-git/ETC5513_Assignment2.git
 * [new branch]      testbranch -> testbranch
branch 'testbranch' set up to track 'origin/testbranch'.

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (testbranch)
$ |
```

Figure 4: Terminal showing branch creation and push

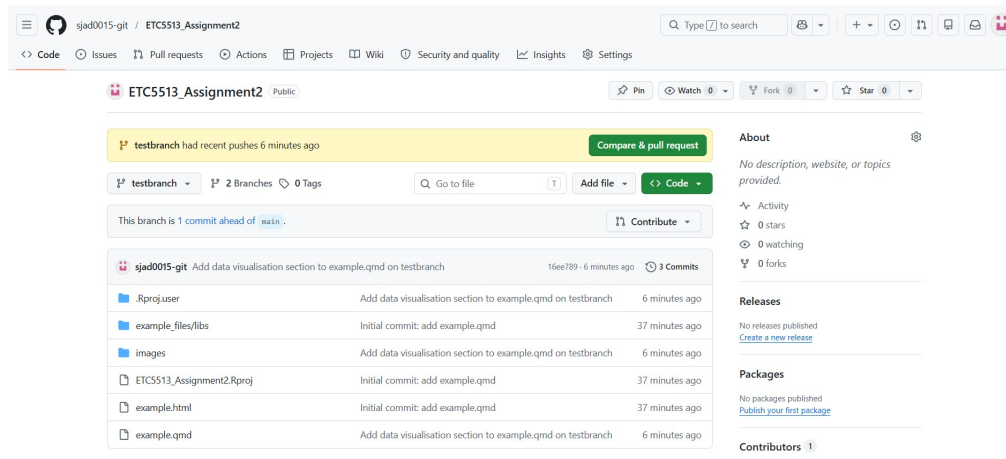


Figure 5: GitHub showing testbranch with new section

Step 4: Add Data Folder and Amend the Previous Commit

I created a data folder in the project and uploaded Assignment 1's dataset in it. Instead of making a new commit, I used the `git commit --amend` command to incorporate the data folder in the last commit:

```
git add .
git commit --amend --no-edit
git push --force origin testbranch
```

When we use `git commit --amend`, it changes the previous commit to reflect the newly added files instead of adding another commit. `--no-edit` prevents the change in the message of the existing commit and makes it useful when one forgets to include a file while committing.

As amending rewrites the commit hash, there will be a discrepancy between the remote and local versions. Therefore, `git push --force` is necessary. This operation should be restricted to a feature branch that belongs to one person and not a common branch such as `main`. The committed files and the data folder on GitHub are shown below.

```

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (testbranch)
$ git add .

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (testbranch)
$ git commit --amend --no-edit
[testbranch 6109ca3] Add data visualisation section to example.qmd on testbranch
Date: Mon May 11 13:06:34 2026 +1000
10 files changed, 183 insertions(+), 8 deletions(-)
create mode 100644 data/36325376_Shravani_Jadhav.csv
create mode 100644 images/Step2a.jpg
create mode 100644 images/Step2b.jpg
create mode 100644 images/Step3a.jpg
create mode 100644 images/Step3b.jpg

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (testbranch)
$ git push --force origin testbranch
Enumerating objects: 31, done.
Counting objects: 100% (31/31), done.
Delta compression using up to 12 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (19/19), 681.73 KiB | 4.48 MiB/s, done.
Total 19 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To github.com:sjad0015-git/ETC5513_Assignment2.git
+ 16ee789...6109ca3 testbranch -> testbranch (forced update)

```

Figure 6: Terminal showing amend commit and force push

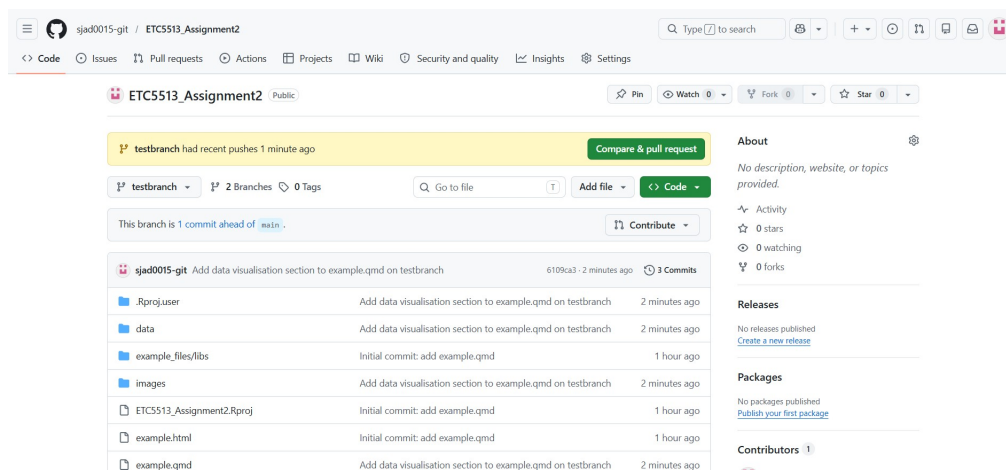


Figure 7: GitHub showing data folder in testbranch

Step 5: Switch to Main and Create a Conflicting Change

I switched back to the main branch using the command `git switch main`. Then, I edited `example.qmd` in the same place where I made changes to `testbranch`, but the content is not the same; I added Speed Analysis instead of Data Visualisation. After that, I staged, committed, and pushed:

```
git switch main
git add .
git commit -m "Add speed analysis section to main"
git push origin main
```

To create a merge conflict, I have set up a scenario that will lead to one. There will be a merge conflict if both branches make changes to the same lines in the same file but in a different way, hence git being unable to decide on what version is kept, and halting the whole process. The images below illustrate the changes made on the main branch.

```
shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (testbranch)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git add .
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/prop/AA839C7F', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/session-b02d7c91/Fc88F3D4', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/session-b02d7c91/Fc88F3D4-contents', LF will be replaced by CRLF the next time Git touches it

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git commit -m "Add speed analysis section to main"
[main b390508] Add speed analysis section to main
6 files changed, 29 insertions(+), 7 deletions(-)
create mode 100644 images/Step4a.jpg
create mode 100644 images/Step4b.jpg

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git push origin main
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 12 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (13/13), 277.24 KiB | 2.12 MiB/s, done.
Total 13 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
To github.com:sjad0015-git/ETC5513_Assignment2.git
  31d088b..b390508  main -> main

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$
```

Figure 8: Terminal showing commit and push on main

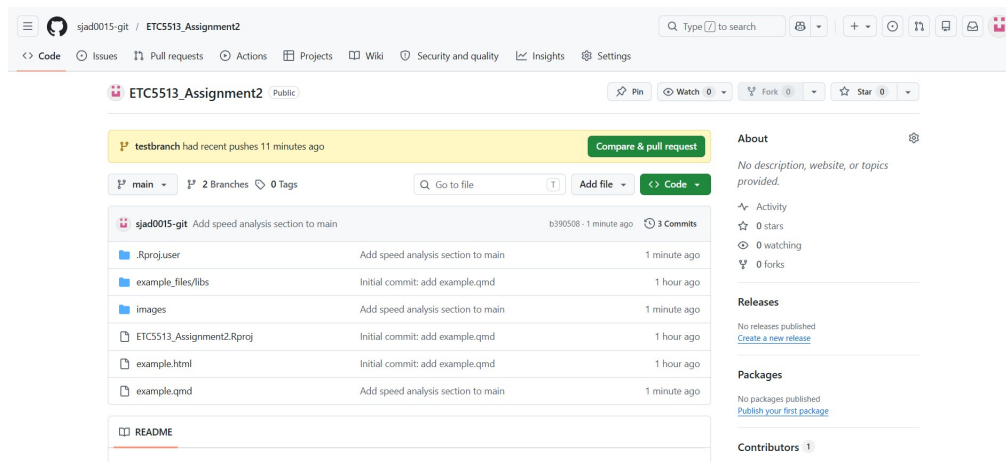


Figure 9: GitHub showing main branch with speed analysis section

Step 6: Merge testbranch into Main and Resolve the Conflict

I merged testbranch into main using `git merge testbranch`. Git detected a conflict in `example.qmd` because both branches had changed the lines. Git added conflict markers to the file to show both versions.

HEAD means the branch, which is main. Everything between `< HEAD` and `=====` is the branch version. Everything between `=====` and `> testbranch` is the version. I edited `example.qmd` in RStudio by keeping both sections, but removed all conflict marker lines. Then I staged the file and completed the merge:

```
git add .
git commit -m "Merge testbranch into main and resolve conflict in example.qmd."
git push origin main
```

The screenshots show conflict markers, in RStudio. They also show the state on GitHub.

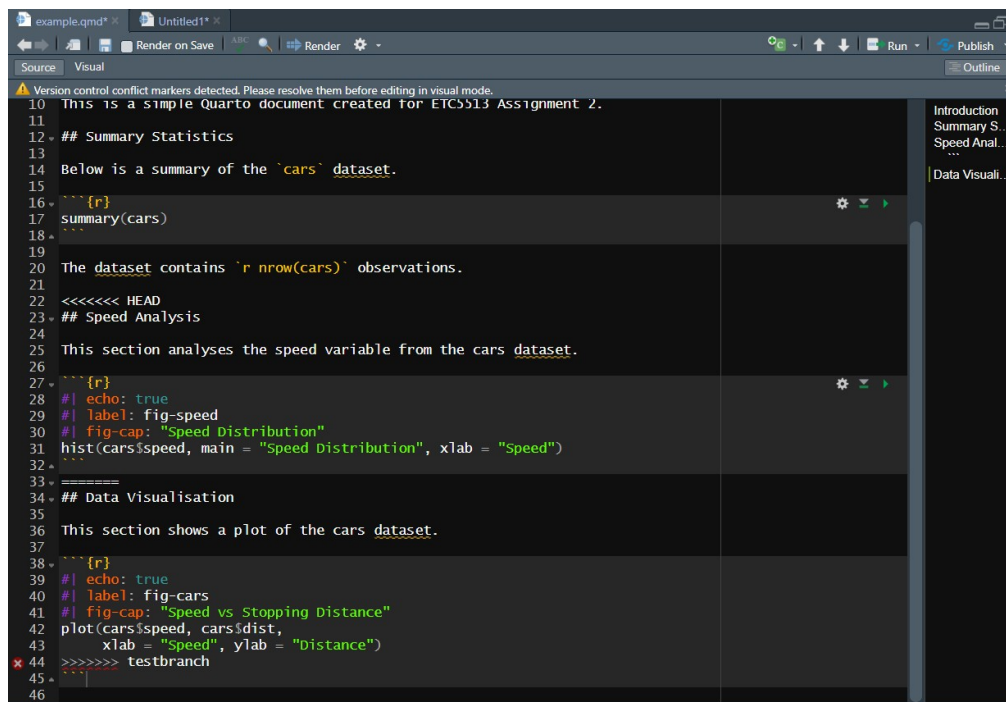


Figure 10: Conflict markers visible in `example.qmd` in RStudio

```

shrav@shrau: ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git merge testbranch
Auto-merging .Rproj.user/D3C41FC8/sources/prop/AA839C7F
CONFLICT (content): Merge conflict in .Rproj.user/D3C41FC8/sources/prop/AA839C7F
Auto-merging .Rproj.user/D3C41FC8/sources/session-b02dc91/FC88F3D4
CONFLICT (content): Merge conflict in .Rproj.user/D3C41FC8/sources/session-b02dc91/FC88F3D4
Auto-merging .Rproj.user/D3C41FC8/sources/session-b02dc91/FC88F3D4-contents
CONFLICT (content): Merge conflict in .Rproj.user/D3C41FC8/sources/session-b02dc91/FC88F3D4-contents
Auto-merging example.qmd
CONFLICT (content): Merge conflict in example.qmd
Automatic merge failed; fix conflicts and then commit the result.
shrav@shrau: ~/OneDrive/Desktop/ETC5513_Assignment2 (main) (MERGING)
$ AC
shrav@shrau: ~/OneDrive/Desktop/ETC5513_Assignment2 (main) (MERGING)
$ git checkout --ours .Rproj.user/D3C41FC8/sources/prop/AA839C7F
git checkout --ours .Rproj.user/D3C41FC8/sources/session-b02dc91/FC88F3D4
git checkout --ours .Rproj.user/D3C41FC8/sources/session-b02dc91/FC88F3D4-contents
Updated 1 path from the index
Updated 1 path from the index
shrav@shrau: ~/OneDrive/Desktop/ETC5513_Assignment2 (main) (MERGING)
$ AC
shrav@shrau: ~/OneDrive/Desktop/ETC5513_Assignment2 (main) (MERGING)
$ git add .
warning: in the working copy of '.Rproj.user/D3C41FC8/pcs/files-pane.pper', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/pcs/source-pane.pper', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/pcs/windowlayoutstate.pper', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/pcs/workbench-pane.pper', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/prop/7E91EE47', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/prop/AA839C7F', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/session-b02dc91/7D54F51D-contents', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/session-b02dc91/FC88F3D4', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/session-b02dc91/FC88F3D4-contents', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/shared/notebooks/paths', LF will be replaced by CRLF the next time Git touches it
shrav@shrau: ~/OneDrive/Desktop/ETC5513_Assignment2 (main) (MERGING)
$ git commit -m "Merge testbranch into main and resolve conflict in example.qmd"
[main f7c7d28] Merge testbranch into main and resolve conflict in example.qmd
shrav@shrau: ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git push origin main
Enumerating objects: 41, done.
Counting objects: 100% (38/38), done.
Delta compression using up to 12 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (15/15), 121.01 KiB | 1.41 MiB/s, done.
Total 15 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (9/9), completed with 8 local objects.
To github.com:ad0015-git/ETC5513_Assignment2.git
   f7c7d28f..f7c7d28f main -> main
shrav@shrau: ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$

```

Figure 11: Terminal showing merge conflict detection, resolution and successful push to GitHub

Step 7: Tag the Commit as v1.0

I made a tag called v1.0 on the current commit of main and I pushed it to GitHub. I used these commands:

```

git tag -a v1.0 -m "Version 1.0: merge complete"
git push origin v1.0
git tag

```

An annotated tag is really useful because it stores extra information than a normal tag. Tags are important because they mark points in a projects history like version releases or report milestones. Normally git push does not push tags so I had to use git push origin v1.0 to share the v1.0 tag on GitHub. I ran git tag to make sure it was created successfully.

The screenshots below show how I created the tag in the terminal and now the v1.0 tag is visible, on GitHub.

```
shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git tag -a v1.0 -m "Version 1.0: merge complete"

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git push origin v1.0
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 188 bytes | 94.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:sjad0015-git/ETC5513_Assignment2.git
 * [new tag]          v1.0 -> v1.0

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git tag
v1.0

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$
```

Figure 12: Terminal showing tag creation and push

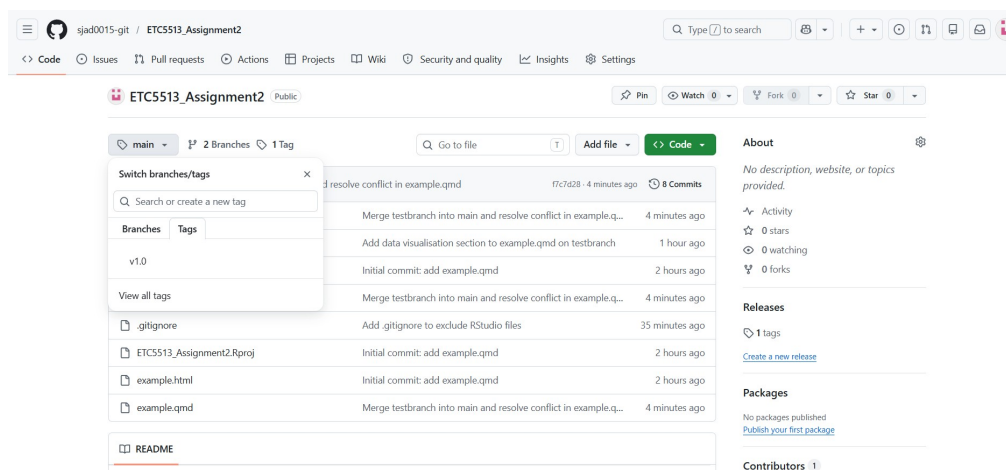


Figure 13: GitHub showing v1.0 tag

Step 8: Delete testbranch Locally and Remotely

After merging testbranch to the main branch successfully, I removed both the local and GitHub repositories of testbranch:

```
git branch -d testbranch
git push origin --delete testbranch
git branch -v
```

Here, the use of lowercase d means safe delete because it will only execute if the branch has been fully merged without losing any work that had not been merged yet. Git push origin

`-delete` is used to remove `testbranch` from GitHub because `git branch -d` can only delete it locally. `Git branch -v` verifies that the branch has been deleted.

Deleting merged branches helps in keeping the repository clean and avoids confusion in knowing which branches have not been deleted yet. The process is shown below.

```
shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git branch -d testbranch
Deleted branch testbranch (was 433427f).

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git push origin --delete testbranch
To github.com:sjad0015-git/ETC5513_Assignment2.git
- [deleted]          testbranch

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git branch -v
* main f7c7d28 Merge testbranch into main and resolve conflict in example.qmd

shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$
```

Figure 14: Terminal showing branch deletion locally and remotely

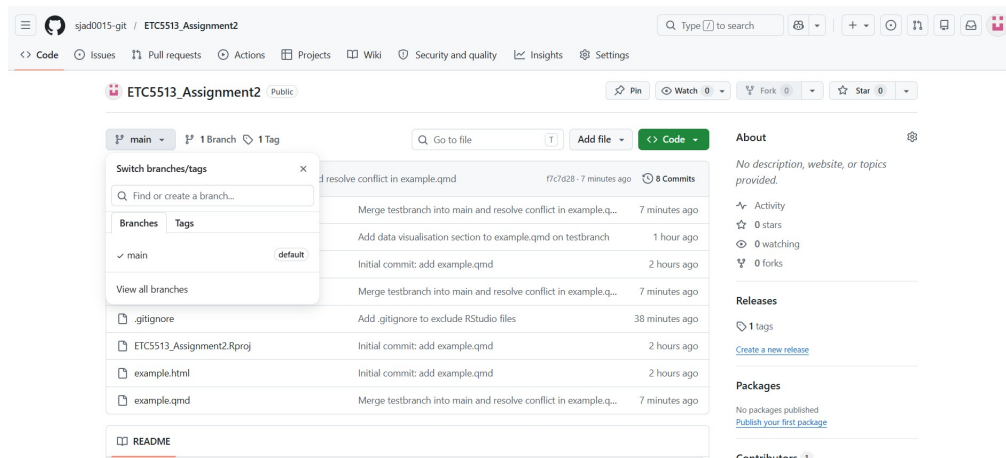


Figure 15: GitHub showing only main branch remains

Step 9: Show the Commit Log in Condensed Form

I used the below command to print out the complete commit history in a concise manner:

```
git log --oneline
```

The above command prints out each commit along with abbreviated commit hash and its message. It provides a concise overview of the whole commit history of the project. Below is the image of the complete commit history of this project.

```
shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git log --oneline
f7c7d28 (HEAD -> main, tag: v1.0, origin/main) Merge testbranch into main and resolve conflict in example.qmd
433427f Add .gitignore to exclude RStudio files
f1638ff Add .gitignore to exclude RStudio files
96b3754 Add screenshots for steps 1-3
b390508 Add speed analysis section to main
6109ca3 Add data visualisation section to example.qmd on testbranch
31d088b added step1 image to images folder
6909860 Initial commit: add example.qmd
shrav@ShrauJ MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$
```

Figure 16: Terminal showing condensed commit log

Step 10: Add a New Plot, Commit, and Undo with git reset --soft

I added an additional section on Pressure Analysis with a graph of the pressure data set in example.qmd. I then performed a commit operation. I then performed git reset --soft HEAD~1 operation, which undid the commit but not the file changes.

```
git add .
git commit -m "Add pressure plot section to example.qmd"
git reset --soft HEAD~1
git status
git log --oneline
```

The git reset --soft HEAD~1 command rolls back the branch one commit but preserves the changes of files as staged changes. Such commands can be very useful in situations where one wants to undo the commit temporarily, without losing any changes in files, such as rewriting the commit message or making other changes before committing. After running git reset --soft, git status confirms the changes are still staged and git log --oneline confirms the commit has been removed from the history.

```

shrav@Shrau: MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git add .
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/prop/AAB39C7F', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/session-b02d7c91/FC88F3D4', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.Rproj.user/D3C41FC8/sources/session-b02d7c91/FC88F3D4-contents', LF will be replaced by CRLF the next time Git touches it

shrav@Shrau: MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git commit -m "Add pressure plot section to example.qmd"
[main 568a5fa] Add pressure plot section to example.qmd
12 files changed, 36 insertions(+), 13 deletions(-)
create mode 100644 images/Step6b.jpg
create mode 100644 images/Step7a.jpg
create mode 100644 images/Step7b.jpg
create mode 100644 images/Step8a.jpg
create mode 100644 images/Step8b.jpg
create mode 100644 images/Step9.jpg

shrav@Shrau: MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git reset --soft HEAD~1

shrav@Shrau: MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   .Rproj.user/D3C41FC8/sources/prop/AAB39C7F
    modified:   .Rproj.user/D3C41FC8/sources/session-b02d7c91/FC88F3D4
    modified:   .Rproj.user/D3C41FC8/sources/session-b02d7c91/FC88F3D4-contents
    modified:   .Rproj.user/shared/notebooks/BC625FD9-example/1/D3C41FC8b02d7c91/chunks.json
    modified:   .Rproj.user/shared/notebooks/BC625FD9-example/1/s/chunks.json
    modified:   example.qmd
    new file:   images/Step6b.jpg
    new file:   images/Step7a.jpg
    new file:   images/Step7b.jpg
    new file:   images/Step8a.jpg
    new file:   images/Step8b.jpg
    new file:   images/Step9.jpg

shrav@Shrau: MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ git log --oneline
f7c7d28 (HEAD -> main, tag: v1.0, origin/main) Merge testbranch into main and resolve conflict in example.qmd
433427f Add .gitignore to exclude RStudio files
a1638ff Add .gitignore to exclude RStudio files
a6b3754 Add screenshots for steps 1-3
b390508 Add speed analysis section to main
6109ca3 Add data visualisation section to example.qmd on testbranch
a10088b added step1 image to images folder
6909860 Initial commit: add example.qmd

shrav@Shrau: MINGW64 ~/OneDrive/Desktop/ETC5513_Assignment2 (main)
$ !

```

Figure 17: Terminal showing commit, reset --soft, git status and git log

Conclusion

This guide has shown how to carry out a complete Git and GitHub workflow ranging from the initialization of a repository to dealing with merge conflicts and undoing commits. In carrying out the workflow, some important aspects of git have been illustrated. All these concepts make up the basis of collaborative and reproducible data analysis. The use of branches, appropriate commit messages, and version tagging makes it easy to collaborate with others on projects. Git ensures that nothing is fully lost and that all operations done are accounted for.